

Your First Nodes

A workspace, a publisher, a subscriber, and five ways to break them

Prof. Anis Koubaa

EE 414 — Introduction to Robotics
College of Engineering
Alfaisal University

Fall 2026 — Week 2, Lecture B



How this session works

You type. I type. We stop at every checkpoint until the room is together.

- 1 A workspace and a package
- 2 A publisher
- 3 Looking at it from outside
- 4 A subscriber
- 5 One command instead of three
- 6 Breaking it on purpose

A workspace and a package

Before anything else

```
$ printenv ROS_DISTRO  
$ printenv ROS_DOMAIN_ID  
$ ros2 topic list
```

Three answers you must see: your distribution name, **your** assigned domain ID, and a list containing `/parameter_events` and `/rosout`.

If ROS_DISTRO is empty

You have not sourced the underlay in this terminal. That is not a broken installation — it is a fresh shell. Source it and try again.

Checkpoint 0. Hand up if any of the three is wrong. Do not go on without it.

Create the workspace

```
$ mkdir -p ~/ee414_ws/src  
$ cd ~/ee414_ws  
$ colcon build  
$ source install/setup.bash
```

Building an empty workspace looks pointless. It is not — it creates `install/` and therefore `install/setup.bash`, which is the file every future terminal will source.

In the field

You will use this workspace for the rest of the semester. Every week adds one package to `src/`. Do not create a new workspace per week — the whole point is that packages accumulate and are used together.

Create the package

```
$ cd ~/ee414_ws/src
$ ros2 pkg create --build-type ament_python \
  --dependencies rclpy std_msgs \
  ee414_w02_first_node
```

<code>--build-type ament_python</code>	a Python package (the alternative is C++)
<code>--dependencies</code>	what it needs; written into the manifest for you

Declaring dependencies now is not bureaucracy. It is how someone else — or you, on the robot in Week 14 — installs what this package needs without guessing.

What it made for you

```
ee414_w02_first_node/  
  package.xml      <- name, version, dependencies  
  setup.py         <- how to install it; ENTRY POINTS live here  
  ee414_w02_first_node/  
    __init__.py    <- your Python module. Code goes HERE.  
  resource/ test/
```

Note the repeated name: the outer folder is the **package**, the inner one is the **Python module**. Your `.py` files go in the inner one.

Under the hood

Putting code in the outer folder is the single most common Week-2 mistake. It builds without complaint and then cannot be found at run time.

A publisher


```
import rclpy
from rclpy.node import Node
from std_msgs.msg import String
class Talker(Node):
    def __init__(self):
        super().__init__('talker')
        self.pub = self.create_publisher(String, 'chatter', 10)
        self.create_timer(0.5, self.tick)      # 2 Hz
        self.i = 0
    def tick(self):
        msg = String()
        msg.data = f'hello {self.i}'
        self.pub.publish(msg)
        self.get_logger().info(f'sent: {msg.data}')
        self.i += 1
```

The four lines that make it runnable

```
def main():  
    rclpy.init()  
    rclpy.spin(Talker())  
    rclpy.shutdown()
```

<code>rclpy.init()</code>	join the graph
<code>rclpy.spin(node)</code>	hand control to ROS 2: wait for events, call your callbacks
<code>rclpy.shutdown()</code>	leave cleanly

Where the loop went

spin is the loop — it blocks forever, waking to run timer and subscriber callbacks. You never write `while True` in ROS 2: you declare what happens on each event, and *spin* calls you.

Tell the build system the node exists

In setup.py:

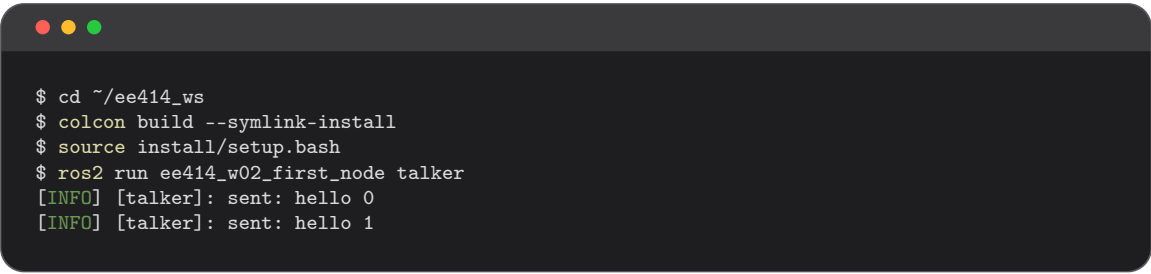
```
entry_points={
    'console_scripts': [
        'talker = ee414_w02_first_node.talker:main',
    ],
}
```

Read it right to left: run main, in module ee414_w02_first_node.talker, as a command called talker.

In the field

The number-one cause of “executable not found” in this course. The name before = is what you type after `ros2 run`; the part after must match your file and function exactly. A typo builds cleanly and fails at run time.

Build it and run it



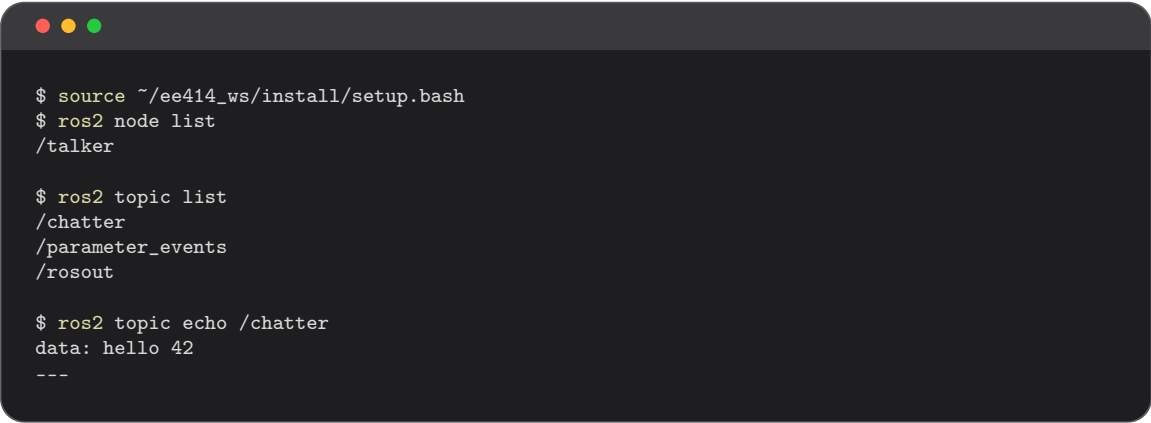
```
$ cd ~/ee414_ws
$ colcon build --symlink-install
$ source install/setup.bash
$ ros2 run ee414_w02_first_node talker
[INFO] [talker]: sent: hello 0
[INFO] [talker]: sent: hello 1
```

`--symlink-install` links your source instead of copying it, so editing a `.py` file takes effect without rebuilding. Use it all semester.

Checkpoint 1. Everyone sees hello counting up. Hands up if not.

Looking at it from outside

Leave it running. Open a second terminal.



```
$ source ~/ee414_ws/install/setup.bash
$ ros2 node list
/talker

$ ros2 topic list
/chatter
/parameter_events
/rosout

$ ros2 topic echo /chatter
data: hello 42
---
```

You added no logging to make this work. The graph is inspectable because it is a graph — this is the property that will save you in Weeks 9 through 11.

Three more commands worth memorising

```
$ ros2 topic hz /chatter
average rate: 2.001      <- is it actually running at the rate you set?

$ ros2 topic info /chatter --verbose
Type: std_msgs/msg/String
Publisher count: 1      <- is anyone publishing at all?

$ ros2 topic pub /chatter std_msgs/msg/String "{data: injected}"
```

The third one publishes *without writing a node*. You can drive a robot from the command line with it — which is how you test a subscriber before its publisher exists.

Checkpoint 2. Everyone has seen `echo`, `hz` and `info`.

A subscriber


```
class Listener(Node):
    def __init__(self):
        super().__init__('listener')
        self.create_subscription(String, 'chatter', self.on_msg, 10)

    def on_msg(self, msg):
        self.get_logger().info(f'got: {msg.data}')
```

Four things must match the publisher: the **type** (String), the **topic name** (chatter), the **QoS** (here, depth 10 — compatible defaults), and the **domain ID**.

Add a second entry point in setup.py, rebuild, and run it in a third terminal.

Two nodes, one graph

```
$ ros2 run ee414_w02_first_node listener  
[INFO] [listener]: got: hello 87  
[INFO] [listener]: got: hello 88  
  
$ rqt_graph
```



Checkpoint 3. You have drawn your first computation graph.

What you just proved to yourself

You did this	Which demonstrates
Started them in either order	No master, no start-up order
Killed the talker; listener survived	A node dies alone
Ran topic echo beside both	A third listener needs no change to the publisher
Neither node names the other	They agree on a topic and a type, nothing else

Four of Lecture A's claims, verified by you in twenty minutes. **Try killing the talker now**, and watch the listener wait patiently rather than crash.

One command instead of three

Launch files

Three terminals for two nodes. A real robot has fifteen. That does not scale, and you cannot reproduce it reliably at 9 a.m. on demo day.

```
from launch import LaunchDescription
from launch_ros.actions import Node

def generate_launch_description():
    return LaunchDescription([
        Node(package='ee414_w02_first_node', executable='talker'),
        Node(package='ee414_w02_first_node', executable='listener'),
    ])
```

In `launch/chatter_launch.py`, plus one line in `setup.py` so the folder is installed. **Note:** this Node is a launch description, not the `rclpy` Node you subclassed. Same word, different thing.

Run the whole system

```
$ colcon build --symlink-install
$ source install/setup.bash
$ ros2 launch ee414_w02_first_node chatter_launch.py
[INFO] [talker-1]: sent: hello 0
[INFO] [listener-2]: got: hello 0
```

One command. Both nodes. Ctrl-C stops both together, and the prefixes tell you which node each line came from.

In the field

Launch files are also where parameters, remappings and namespaces are set — which is how the same node runs twice, on two robots, with different topic names and no code change. Week 8 needs this; Week 11 is impossible without it.

Breaking it on purpose

Five failures — cause them, read the symptom, fix them

Do this	Symptom	The lesson
New terminal, do not source	Package not found	Sourcing is per shell
Misspell the entry point	No executable found	The name is a mapping, not magic
Change the listener's domain ID	Both run. Silence.	Discovery is partitioned
Rename the topic in one node only	Both run. Silence.	Names are the whole contract
Set the listener QoS to BEST_EFFORT, publisher RELIABLE	Both run. Silence.	QoS must be compatible

Three of the five look identical from the outside: two healthy nodes, no data.

The diagnostic order that separates them

```
$ ros2 node list           # are both alive?
$ ros2 topic list         # does the topic exist?
$ ros2 topic info /chatter --verbose # counts, types, QoS
$ printenv ROS_DOMAIN_ID  # ... in BOTH terminals
```

Publisher count 1, subscriber 0	The subscriber is on another topic or domain
Both counts 1, no data	QoS mismatch — read the profiles
Topic missing entirely	The publisher is not running, or not sourced

Learn this order now. It is the same four commands in Week 11, on a robot with forty topics.

Before you leave, and before Week 3

Leaving this room, you should have: a workspace, a package, a talker, a listener, a launch file that starts both, and five failures you have caused and fixed.

Assignment 1 is posted today, due Week 3. It extends this package: a publisher at a rate you choose, a subscriber that computes something from what it receives, and a launch file that starts the pair with parameters.

The habit to build now

Commit your workspace to Git today, and commit at the end of every session. In Week 14 your project mark depends partly on a repository history that shows three people working. That history cannot be created retroactively.

Next week: services, parameters, and custom message types.